Презентация по лабораторной работе 7

Однократное гаммирование

Ерфан Хосейнабади

Российский университет дружбы народов, Москва, Россия

Section 1

Информация

Section 2

Цель работы

Цель работы

Освоить на практике применение режима однократного гаммирования — метода симметричного шифрования, основанного на операции XOR. Научиться решать две криптографические задачи: нахождение шифротекста по известному ключу и открытому тексту, а также восстановление ключа по известным шифротексту и открытому тексту.

Section 3

Выполнение лабораторной работы

Схема однократного гаммирования Вернама

Гаммирование представляет собой наложение (снятие) на открытые (зашифрованные) данные последовательности элементов других данных, полученной с помощью некоторого криптографического алгоритма.

```
File  Machine  View  Input  Devices  Help
  GNU nano 5.6.1
#!/bin/bash

echo "==========================================="
echo "LAB 7: One-Time Gamma Encryption"
echo "==========================================="

# ----------------------------------------------------------------
# TASK 1: Find ciphertext from known plaintext + known key
# ----------------------------------------------------------------
echo ""
echo "--- TASK 1: Encrypt (Find ciphertext) ---"

# Given plaintext (from lab page 2)
PLAINTEXT="Штирлиц - Вы Герой!!"
echo "Plaintext: $PLAINTEXT"

# Given key (from lab page 2)
KEY_HEX="050C177F0E4E37D29410092E2257FFC80BB27054"
echo "Key (hex): $KEY_HEX"

# Convert plaintext to CP1251 hex (as used in lab)
PLAIN_HEX=$(echo -n "$PLAINTEXT" | iconv -f UTF-8 -t CP1251 | xxd -p | tr -d '\n')
echo "Plaintext (hex): $PLAIN_HEX"

# XOR to get ciphertext
python3 << EOF
plain = bytes.fromhex("$PLAIN_HEX")
key = bytes.fromhex("$KEY_HEX")
cipher = bytes([plain[i] ^ key[i] for i in range(len(plain))])
print(f"Ciphertext (hex): {cipher.hex().upper()}")
```

Laptop battery low
Approximately **41 minutes** remaining (20%)

Операция XOR

Операция сложения по модулю 2 (XOR, обозначается) лежит в основе однократного гаммирования:

- 0 0 = 0
- 0 1 = 1
- 1 0 = 1
- 1 1 = 0

Метод является симметричным: двойное прибавление одной и той же величины восстанавливает исходное значение.

Задание 1: Нахождение шифротекста

Известны: - Открытый текст: `Штирлиц – Вы Герой!!` - Ключ (hex): `050C177F0E4E37D29410092E2257FFC80BB27054`

Необходимо найти шифротекст по формуле: [C_i = P_i K_i]

```
erfan@rocky:~$ bash lab.sh
==========================================
LAB 7: One-Time Gamma Encryption
==========================================

--- TASK 1: Encrypt (Find ciphertext) ---
Plaintext: Штирлиц – Вы Герой!!
Key (hex): 050C177F0E4E37D29410092E2257FFC80BB27054
Plaintext (hex): d8f2e8f0ebe8f6209620c2fb20c3e5f0eee92121
Ciphertext (hex): DDFEFF8FE5A6C1F20230CBD502941A38E55B5175

This should match lab's ciphertext: DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75
```

Laptop battery low
Approximately **41 minutes** remaining (20%)

Рис. 2: Скрипт для выполнения первого задания

Результат первого задания

Открытый текст в кодировке CP1251 (hex): `d8f2e8f0ebe8f6209620c2fb20c3e5f0ee692121`

Вычисленный шифротекст (hex): `DDFEF8FE5A6C1F20230CBD502941A38E55B5175`

File Machine View Input Devices Help

GNU nano 5.6.1

Laptop battery low
Approximately **41 minutes** remaining (20%)

```
# TASK 2: Find key to get "С Новым Годом, друзья!
# ----------------------------------------------
echo ""
echo "--- TASK 2: Find key for target message ---"

# Given ciphertext (Müller's from lab page 2)
CIPHER_HEX="DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175"
echo "Ciphertext (hex): $CIPHER_HEX"

# Target plaintext
TARGET="С Новым Годом, друзья!"
echo "Target plaintext: $TARGET"

# Convert target to CP1251 hex
TARGET_HEX=$(echo -n "$TARGET" | iconv -f UTF-8 -t CP1251 | xxd -p | tr -d '\n')
echo "Target (hex): $TARGET_HEX"

# XOR to find key - FIXED to handle length mismatch
python3 << EOF
cipher = bytes.fromhex("$CIPHER_HEX")
target = bytes.fromhex("$TARGET_HEX")

# Use the minimum length to avoid index error
min_len = min(len(cipher), len(target))
key = bytes([cipher[i] ^ target[i] for i in range(min_len)])

print(f"Found key (hex): {key.hex().upper()}")
print(f"Key length: {len(key)} bytes")
print(f"Key (bytes): {key}")
```

Задание 2: Восстановление ключа

Известны: - Шифротекст (hex): `DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175` - Целевое сообщение: `С Новым Годом, друзья!`

Ключ восстанавливается по формуле: [K_i = C_i P_i]

```
--- TASK 2: Find key for target message ---
Ciphertext (hex): DDFEFF8FE5A6C1F2B930CBD502941A38E55B5175
Target plaintext: С Новым Годом, друзья!
Target (hex): d120cdeee2fbec20c3eee4eeec2c20e4f0f3e7fcff21
Found key (hex): 0CDE3261075D2DD27ADE2F3BEEB83ADC15A8B689
Key length: 20 bytes
Key (bytes): b'\x0c\xde2a\x07]-\xd2z\xde/;\xee\xb8:\xdc\x15\xa8\xb6\x89'

WARNING: Length mismatch!
  Ciphertext length: 20 bytes
  Target length: 22 bytes
  Only first 20 bytes of key recovered

==========================================
LAB COMPLETE
==========================================
```

Рис. 4: Скрипт для выполнения второго задания

Проблема несовпадения длин

При выполнении возникла проблема: - Длина шифротекста: 20 байт - Длина целевого сообщения в CP1251: 22 байта

Решение: ограничение по минимальной длине позволило восстановить первые 20 байт ключа.

Результат второго задания

- Целевое сообщение в CP1251 (hex): `d120cdeee2fbec20c3eee4eeec2c20e4f0f3e7fcf21`
- Восстановленный ключ (hex): `0CDE3261075D2DD27ADE2F3BEEB83ADC15A8B689`
- Длина ключа: 20 байт (восстановлена частично)

Также было получено предупреждение о несовпадении длин: только первые 20 байт ключа восстановлены.

Математические формулы

Шифрование: [C_i = P_i K_i]

Расшифрование: [P_i = C_i K_i]

Восстановление ключа: [K_i = C_i P_i]

Где: - (C_i) — i-й символ шифротекста - (P_i) — i-й символ открытого текста - (K_i) — i-й символ ключа

Условия абсолютной стойкости шифра (К. Шеннон)

1. **Полная случайность ключа** — ключ должен быть фрагментом истинно случайной двоичной последовательности
2. **Равенство длин ключа и открытого текста** — ключ не должен быть короче сообщения
3. **Однократное использование ключа** — каждый ключ используется только один раз

Преимущества однократного гаммирования

- **Абсолютная стойкость** — при соблюдении всех условий шифр невозможно взломать
- **Простота реализации** — используется только операция XOR
- **Симметричность** — шифрование и расшифрование выполняются одинаково
- **Отсутствие информации об открытом тексте** — при известном шифротексте все возможные сообщения равновероятны

Недостатки однократного гаммирования

- **Проблема распространения ключей** — нужен надёжный канал для передачи ключа той же длины, что и сообщение
- **Одноразовость** — ключи нельзя использовать повторно
- **Длина ключа** — ключ должен быть не короче сообщения
- **Требование к случайности** — необходимы генераторы истинно случайных чисел

Section 4

Выводы

Выводы

В результате выполнения лабораторной работы я освоил на практике применение режима однократного гаммирования. Были решены две основные криптографические задачи:

1. Вычисление шифротекста по известным открытому тексту и ключу
2. Восстановление ключа по известным шифротексту и открытому тексту

Было подтверждено, что при несовпадении длин ключа и сообщения невозможно полное восстановление ключа, что демонстрирует важность соблюдения условий абсолютной стойкости шифра по Шеннону.